

Digital Signatures

Number-Theoretic Foundations and Applications

July 5, 2026

Outline

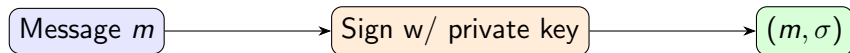
- 1 Motivation
- 2 Number Theory Toolkit
- 3 RSA Signatures
- 4 Discrete-Log Signatures
- 5 Applications
- 6 Conclusion

What Problem Are We Solving?

Goal

Given a message m and a private key, produce σ such that *anyone* with the public key can confirm:

- **Authenticity** — only the key owner could have produced σ
- **Integrity** — any change to m breaks verification
- **Non-repudiation** — the signer can't later deny it



The Group $\mathbb{Z}_n^*$

- $\mathbb{Z}_n^* = \{a \in \{0, \dots, n-1\} : \gcd(a, n) = 1\}$
- Forms a group under multiplication mod n , order $\varphi(n)$
- If $n = p$ prime: $\varphi(p) = p - 1$
- If $n = pq$, distinct primes: $\varphi(n) = (p - 1)(q - 1)$

Why we care

This group structure is the entire “state space” RSA arithmetic lives in.

Fermat & Euler: The Two Load-Bearing Theorems

Fermat's Little Theorem

$$p \text{ prime, } p \nmid a \implies a^{p-1} \equiv 1 \pmod{p}$$

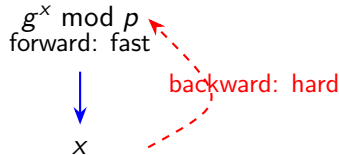
Euler's Theorem

$$\gcd(a, n) = 1 \implies a^{\varphi(n)} \equiv 1 \pmod{n}$$

Everything RSA does is exploit this single congruence.

Discrete Logarithm Problem

- Cyclic group G , generator g , order q
- **Easy**: given x , compute g^x (repeated squaring, $O(\log x)$)
- **Hard**: given $h = g^x$, recover x
- No efficient classical algorithm known in $\mathbb{Z}_p^*$ or elliptic curve groups



RSA: Key Generation

- 1 Pick large primes p, q ; let $n = pq$
- 2 $\varphi(n) = (p - 1)(q - 1)$
- 3 Pick e with $\gcd(e, \varphi(n)) = 1$
- 4 Solve for $d \equiv e^{-1} \pmod{\varphi(n)}$

Keys

Public: (n, e)

Private: d

Sign: $\sigma \equiv m^d \pmod{n}$

Verify: $\sigma^e \bmod n \stackrel{?}{=} m$

Theorem

$(m^d)^e \equiv m \pmod{n}$ for all $m \in \mathbb{Z}_n^*$.

Proof sketch: $ed = 1 + k\varphi(n)$ for some integer k , so

$$(m^d)^e = m^{1+k\varphi(n)} = m \cdot (m^{\varphi(n)})^k \stackrel{\text{Euler}}{=} m \cdot 1^k = m.$$

RSA: Fully Worked Example

$$p = 61, q = 53 \Rightarrow n = 3233, \varphi(n) = 3120$$

$$e = 17 \Rightarrow d = 2753 \quad (17 \times 2753 = 46801 = 15 \times 3120 + 1)$$

Sign $m = 65$:

$$\sigma = 65^{2753} \bmod 3233 = 2790$$

Verify:

$$2790^{17} \bmod 3233 = 65 \checkmark$$

Tamper test

Flip $m \rightarrow 66$: verification gives $65 \neq 66$ — rejected!

Let's generate keys, sign, and verify live.

Call out two small primes and a message number —
we'll watch the arithmetic resolve on screen.

(Switch to the interactive RSA signature tool now.)

ElGamal Signatures

Public key $y = g^x \bmod p$; private key x . **Sign** (pick random ephemeral k , $\gcd(k, p-1) = 1$):

$$r = g^k \bmod p, \quad s = (m - xr)k^{-1} \bmod (p-1)$$

Verify:

$$g^m \stackrel{?}{\equiv} y^r r^s \pmod{p}$$

Critical rule

Never reuse k ! Reuse leaks x instantly via linear algebra.

DSA and ECDSA

- **DSA**: works in a prime-order subgroup of $\mathbb{Z}_p^*$ $\rightarrow$ shorter signatures
- **ECDSA**: replaces $\mathbb{Z}_p^*$ with points on an elliptic curve $E(\mathbb{F}_p)$
- Same underlying idea: DLP hardness, just in a different group

	128-bit security	Signature size
RSA	3072-bit n	large
ECDSA	256-bit curve	small

Where This Shows Up Every Day

- **TLS/HTTPS** — certificate chains signed by CAs
- **Code signing** — OS/app-store update verification
- **Blockchain** — every Bitcoin/Ethereum transaction is ECDSA-signed
- **e-Government & PDF signing** — legal non-repudiation
- **Software supply chain** — PGP/GPG, Sigstore

Takeaways

- Digital signatures = number theory with a trapdoor
- RSA: security from factoring; correctness from Euler's theorem
- ElGamal/DSA/ECDSA: security from discrete logs
- Elliptic curves buy the same security with far smaller keys
- The math you just saw runs behind every HTTPS padlock and blockchain transaction

Questions?